

OPEN SOURCE TECHNICAL SPECIFICATION & MANUAL

AutoFormFiller

A High-Performance, Privacy-Focused Browser Extension and Local Python Daemon for Intelligent Multi-Section Form Automation & Resume Extraction.

SOFTWARE VERSION

v1.1.0 (Production Hardened)

TARGET ENVIRONMENTS

Google Chrome, Brave, MS Edge (Chromium MV3)

SOFTWARE LICENSE

MIT Open Source License

LEAD AUTHOR & ARCHITECT

Aditya Chauhan (Full-Stack Developer)

1. Executive Summary & Purpose

Job applications, university registrations, and enterprise vendor forms require users to repeatedly re-type the exact same information across dozens of fragmented interfaces. While legacy browser autofills support basic names and addresses, they consistently fail on complex career inputs such as current company, work tenure dates, university degrees, graduation batches, high school records, language proficiency levels, and structured technical skills.

AutoFormFiller solves this by pairing a lightweight, secure Chrome Manifest V3 extension with a local, zero-cloud Python daemon backed by SQLite. It provides two non-destructive input workflows:

- **Direct Resume Ingestion:** Drag and drop any `.pdf`, `.docx`, or `.txt` resume. The system extracts career, education, and language details using pattern-matched section analyzers.
- **Manual Profile Management:** Full control to add, edit, override, or append custom field-value pairs into a private local SQLite database.
- **One-Click Page Automation:** When navigating any job portal or registration form, the user selects their profile and clicks *"Auto Fill This Form"* to populate inputs across the DOM instantly.

2. System Architecture

The system follows a decoupled, privacy-first local client-server pattern. Sensitive data never leaves the user's physical machine.

Component	File Path	Core Responsibilities
Service Worker	<code>extension/background.js</code>	Routes messages between popup, content script, and the authenticated localhost API. Uses <code>activeTab</code> for on-demand injection.
Content Script	<code>extension/content.js</code>	Traverses target DOM, enforces strict anti-honeypot visibility checks, maps input tags, and dispatches native browser events.
User Interface	<code>extension/popup.html</code> , <code>popup.js</code>	3-tab modern UI (Fill, Profiles, Add). Safe DOM construction, active domain verification badge, and file upload handler.
REST API Daemon	<code>backend/app.py</code>	Flask HTTP server on <code>127.0.0.1:5000</code> . Enforces <code>X-AFF-KEY</code> token security, scoped CORS, and heuristic field matching.
Resume Engine	<code>backend/resume_parser.py</code>	Multi-section document parser extracting company, dates, university, degree, school, languages, and technical skills.
Database Layer	<code>backend/db.py</code>	SQLite database storing profile metadata and relational key-value attributes with cascade deletions.

3. Step-by-Step User & Setup Guide

3.1. Setting Up the Local Python Backend (Virtual Environment)

AutoFormFiller isolates its dependencies within a standard Python virtual environment (`venv`). Open your terminal and run:

```
# 1. Navigate to the project root directory
cd /home/ADIXPRI/Desktop/autoformfiller

# 2. Initialize the Python virtual environment
python3 -m venv venv

# 3. Activate the environment
source venv/bin/activate

# 4. Install backend dependencies
pip install -r backend/requirements.txt
```

3.2. Launching the Backend Daemon

With the virtual environment active, run the Flask server:

```
python backend/app.py
```

The server initializes on `http://127.0.0.1:5000` . It will automatically create the SQLite database file `backend/autoformfiller.db` upon first launch. Verify it is running by checking `http://127.0.0.1:5000/api/health` .

3.3. Installing the Extension in Google Chrome / Chromium

1. Launch Google Chrome, Brave, or Microsoft Edge.
2. Navigate to the extensions manager by typing `chrome://extensions/` in the URL bar.
3. Enable the **Developer mode** toggle located in the upper-right corner.
4. Click the **Load unpacked** button in the top toolbar.
5. Select the folder: `/home/ADIXPRI/Desktop/autoformfiller/extension` .
6. Pin the **AutoFormFiller** icon to your browser toolbar for quick one-click access.

3.4. End-to-End Usage Walkthrough

1. **Create or Ingest Profile:** Click the extension icon and navigate to the *Add Profile* tab. Drop your resume PDF or type manual fields. Click *Save Profile*.
2. **Open Application Form:** Open any web form, or test locally with the bundled test file:

```
# In Chrome, press Ctrl + O and open:
/home/ADIXPRI/Desktop/autoformfiller/demo_form.html
```

3. **Auto Fill:** Click the extension icon on the form page, select your saved profile, and click **Auto Fill This Form**. All matching visible fields will be populated with animated visual highlights.

4. Security Architecture & Threat Mitigations

Browser extensions communicating with localhost servers present specific security challenges. AutoFormFiller was hardened to defend against drive-by web attacks, CSRF, honeypot exfiltration, and DoS.

Security Defense 1: Authenticated API Token & Scoped CORS (Anti-Drive-By CSRF)

The Threat: If a localhost API allows unauthenticated access or wildcard CORS (`*`), any malicious webpage a user visits could execute `fetch("http://127.0.0.1:5000/api/profiles")` to steal stored resumes.

The Mitigation: `backend/app.py` strictly scopes CORS to extension schemes (`chrome-extension://*`) and localhost. Additionally, it enforces an authenticated pre-shared security token via the `X-AFF-KEY` header on all endpoints. Any regular website attempting to make requests without the header or with custom headers triggers a browser CORS preflight that is instantly blocked by the browser's Same-Origin Policy.

Security Defense 2: Anti-Honeypot & Invisible Field Protection

The Threat: Deceptive job application sites can embed hidden input tags off-screen (e.g., `left: -9999px`, `opacity: 0`, or `1px x 1px`) named `phone`, `address`, or `university` to covertly siphon private profile attributes.

The Mitigation: `extension/content.js` contains the `isSecurelyVisible()` validator. It inspects layout bounding boxes (`rect.width ≥ 10`, `rect.height ≥ 10`), checks viewport coordinates to ensure fields are not placed off-screen, inspects CSS clipping and opacity, and strictly ignores password inputs, hidden inputs, and elements marked `aria-hidden="true"`.

Security Defense 3: Principle of Least Privilege in Permissions

The Threat: Overprivileged extensions requesting `<all_urls>` host permissions have permanent background access to read every tab the user browses.

The Mitigation: In `extension/manifest.json`, `<all_urls>` was removed from host permissions. Network requests are locked down strictly to `http://127.0.0.1:5000/*`. DOM access is scoped to the user-invoked tab via the secure Chrome `activeTab` API.

Security Defense 4: Upload Size Limits & Filename Sanitization

The Threat: Arbitrary file uploads can lead to memory exhaustion (DoS) or path traversal attacks.

The Mitigation: Enforces `MAX_CONTENT_LENGTH = 10MB`, validates against a file extension whitelist (`.pdf`, `.docx`, `.doc`, `.txt`), and sanitizes filenames using `werkzeug.utils.secure_filename`.

5. Developer Guide: How to Extend & Add New Features

AutoFormFiller was engineered with modular abstractions so software engineers can easily extend field mappings, add custom resume extractors, or build adapters for proprietary form components.

5.1. Adding New Form Field Mappings

All heuristic field detection is defined in the `FIELD_MAPPING` dictionary inside `backend/app.py`. To add support for a new form concept (e.g. *Expected Salary* or *Notice Period*):

```
# In backend/app.py → FIELD_MAPPING
FIELD_MAPPING = {
    # Existing fields...
    "salary_expectation": [
        "salary", "expected_salary", "ctc", "expected_ctc",
        "compensation", "remuneration", "target_pay"
    ],
    "notice_period": [
        "notice_period", "notice", "availability",
        "joining_time", "start_immediately", "days_notice"
    ],
}
```

When a webpage form field has an `id`, `name`, `placeholder`, or `label` containing any of these keywords, the backend will automatically map the stored profile value to it.

5.2. Adding New Parsing Rules to the Resume Engine

The resume parser lives in `backend/resume_parser.py`. To add extraction logic for a new field:

```
# In backend/resume_parser.py → _extract_fields_from_text()
def _extract_fields_from_text(text: str) → dict:
    fields = {}
    # ... existing extraction logic ...

    # Example: Custom Notice Period Regex
    notice_match = re.search(r"(\d+)\s*(?:days?|months?)\s*notice", text, re.IGNORECASE)
    if notice_match:
        fields["notice_period"] = notice_match.group(0).strip()

    return fields
```

5.3. Supporting Custom Form Components (e.g., Shadow DOM, Custom UI Libraries)

If you encounter custom web components (such as Salesforce, Workday, or Greenhouse custom widgets), extend the element scanner in `extension/content.js`:

```
// In extension/content.js → collectFormFields()
function collectFormFields() {
  const selectors = [
    'input:not([type="hidden"]):not([type="password"])',
    'textarea',
    'select',
    // Add custom component tags:
    'div[contenteditable="true"]',
    'custom-select-input'
  ].join(",");

  // Extract descriptors and return to background script
}
```

6. Open Source License (MIT License)

AutoFormFiller is licensed as open-source software under the permissive terms of the **MIT License**.

MIT License

Copyright (c) 2026 Aditya Chauhan

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

7. Lead Author & Developer Profile

Aditya Chauhan

Full-Stack Developer | Python · Django · React.js · Next.js · Node.js

Software Engineer with specialized experience developing and shipping high-performance web applications, automation microservices, and end-to-end full-stack architectures. Architect of the AutoFormFiller platform, built to deliver secure, zero-cloud personal data automation for modern web workflows.

EMAIL ADDRESS

suryachauhan367367@gmail.com

CONTACT NUMBER

+91 8369899103

LINKEDIN PROFILE

linkedin.com/in/aditya-chauhan-1b1a95228

GITHUB REPOSITORY & PROFILE

github.com/Aditya367367

LOCATION

Mumbai, Maharashtra, India

CONTRIBUTIONS & PRs

Open for Community Pull Requests